МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

РЕФЕРАТ

з дисципліни: «Архітектура комп'ютерних систем та основи схемотехніки»

на тему:

«Системи числення, двійкова арифметика та принципи машинного представлення даних в обчислювальних системах»

Виконав:
студент групи КН-21
Іваненко Іван Іванович

Перевірив:
доцент кафедри ОТ, к.т.н.
Петренко Петро Петрович

Київ — 2026

ЗМІСТ

ВСТУП **3**

РОЗДІЛ 1. ЗАГАЛЬНІ ВІДОМОСТІ ПРО СИСТЕМИ ЧИСЛЕННЯ **4**

1.1. Поняття системи числення. Непозиційні та позиційні системи **4**

1.2. Базові характеристики позиційних систем числення **5**

РОЗДІЛ 2. ОСНОВНІ КОМП'ЮТЕРНІ СИСТЕМИ ЧИСЛЕННЯ **7**

2.1. Двійкова система числення (Binary) як основа цифрової електроніки **7**

2.2. Вісімкова (Octal) та шістнадцяткова (Hexadecimal) системи **8**

РОЗДІЛ 3. АЛГОРИТМИ ПЕРЕВОДУ ЧИСЕЛ ТА ДВІЙКОВА АРИФМЕТИКА **9**

3.1. Переведення цілих та дробових чисел. Дроби та IEEE 754 ..... **9**

3.2. Швидкий перехід між системами (метод тріад і тетрад) **12**

3.3. Арифметичні операції та машинне кодування чисел **13**

ВИСНОВКИ **17**

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ **18**

ВСТУП

Актуальність теми. Сучасна обчислювальна техніка, попри колосальну складність архітектур процесорів, базується на концепціях середини XX століття. В основі будь-якого комп'ютера — від мікроконтролера до суперкомп'ютера — лежить оперування дискретними сигналами. Вибір систем числення для внутрішнього представлення даних та людино-машинної взаємодії є прямим наслідком законів фізики напівпровідників, математичної логіки та схемотехніки.

З точки зору схемотехніки, створення елементної бази на 10 стійких рівнів напруги призводить до критичного падіння завадостійкості та ускладнення схем. Навпаки, дворівнева логіка (логічна «1» або «0») реалізується мінімальною кількістю транзисторів, забезпечуючи граничну швидкодію та максимальну надійність. Тому базисом цифрової техніки стала **двійкова система числення (Binary)**, а для компактного запису машинних слів і адрес пам'яті виникли **вісімкова (Octal)** та **шістнадцяткова (Hexadecimal)** системи.

Мета реферату — всебічно дослідити теоретичні засади позиційних систем числення, розглянути двійкову, вісімкову та шістнадцяткову системи в контексті комп'ютерної схемотехніки, алгоритми прямого та зворотного переводу чисел, специфіку двійкової арифметики та формати кодування цілих чисел зі знаком (прямий, обернений та додатковий коди).

Завдання дослідження:

1. Розкрити відмінності між непозиційними та позиційними системами, вивести поліноміальну розгорнуту форму числа.
2. Проаналізувати фізичні, математичні та апаратні причини домінування двійкової системи в цифровій техніці.
3. Охарактеризувати вісімкову та шістнадцяткову системи як компактні людино-орієнтовані подання бітових послідовностей.
4. Дослідити алгоритми переводу чисел (ділення, множення, схема Горнера, тетради) та природу похибок у форматі IEEE 754.
5. Опанувати двійкове додавання, множення у стовпчик та довести переваги додаткового коду для заміни віднімання додаванням.

РОЗДІЛ 1. ЗАГАЛЬНІ ВІДОМОСТІ ПРО СИСТЕМИ ЧИСЛЕННЯ

1.1. Поняття системи числення. Непозиційні та позиційні системи

Система числення — це узгоджена сукупність прийомів, правил та символьних позначень, призначена для однозначного запису (кодування), іменування та арифметичної обробки кількісних величин [6]. Символи, що використовуються для запису чисел, називають **цифрами**, а їх повну множину — **алфавітом системи числення**.

Історично всі системи числення поділяються на два принципово відмінні класи: **непозиційні** та **позиційні**.

Непозиційні системи числення

У непозиційних системах кількісне значення (вага), що приписується цифрі, **не залежить від її розташування (позиції)** у числовому записі. Найпростішим прикладом є так звана одинична (унарна) система числення, де кількість предметів фіксувалася нанесенням відповідного числа засічок на дереві чи кістці.

Класичним прикладом розвиненої непозиційної системи є **римська система числення**, що використовує латинські літери:

- $\mathrm{I} = 1$
- $\mathrm{V} = 5$
- $\mathrm{X} = 10$
- $\mathrm{L} = 50$
- $\mathrm{C} = 100$
- $\mathrm{D} = 500$
- $\mathrm{M} = 1000$

Значення числа в римській системі визначається простим сумуванням значень цифр (якщо менша цифра стоїть праворуч від більшої: $\mathrm{VI} = 5 + 1 = 6$) або відніманням (якщо менша цифра передує більшій: $\mathrm{IV} = 5 - 1 = 4$).

Критичні недоліки непозиційних систем:

1. **Відсутність концепції нуля** як повноправної цифри, що позначає відсутність розряду або порожню величину.
2. **Неможливість запису дробових або скільки завгодно великих чисел** без введення нескінченної множини нових символів.
3. **Надзвичайна складність виконання базових арифметичних операцій**. Виконання множення або ділення «у стовпчик» у римській нотації (наприклад, $\mathrm{MCMLXXXIV} \times \mathrm{LXXVII}$) алгоритмічно настільки громіздке, що вимагало використання спеціальних лічильних дощок (абаків) і спеціальної багаторічної підготовки рахівників.

Позиційні системи числення

Революційним стрибком у розвитку математики та обчислювальної культури став винахід **позиційних систем числення** (зародилися в Стародавньому Вавилоні у шістдесятковому вигляді, остаточно сформувалися у десятковій формі в Індії і були популяризовані арабськими математиками, зокрема Аль-Хорезмі).

> ***Означення:*** *Система числення називається* ***позиційною****, якщо одна й та сама цифра має різне вагове значення залежно від того, яке саме місце (розряд) вона посідає в загальному рядку символів числа.*

Наприклад, у десятковому числі 777 перша сімка праворуч означає сім простих одиниць ($7 \times 10^0 = 7$), друга сімка — сім десятків ($7 \times 10^1 = 70$), а третя — сім сотень ($7 \times 10^2 = 700$). Вага розряду експоненціально зростає зі зміщенням уліво.

1.2. Базові характеристики позиційних систем числення

Кожна позиційна система числення однозначно характеризується трьома фундаментальними параметрами:

1. **Основа системи числення (b, radix / base)** — ціле додатне число ($b \geq 2$), що дорівнює кількості елементарних цифр в алфавіті даної системи, а також показує, у скільки разів одиниця наступного (старшого) розряду більша за одиницю поточного розряду.
2. **Алфавіт системи числення (A_b)** — упорядкований набір з b унікальних знаків (цифр):

   $$A_b = \{d_0, d_1, d_2, \ldots, d_{b-1}\}$$

   Для систем з основою $b \leq 10$ використовуються арабські цифри $0, 1, \ldots, b-1$. Якщо основа $b > 10$, до набору цифр додають великі літери латинського алфавіту ($\mathrm{A} = 10, \mathrm{B} = 11, \mathrm{C} = 12$ тощо).
3. **Розрядна вага** — множник виду b^k, де k — цілий індекс позиції розряду, відрахований відносно коми (роздільника цілої та дробової частини).

Розгорнута форма запису числа

У довільній позиційній системі числення з основою b довільне дійсне число N, записане у згорнутому символьному вигляді як:

$$N = (a_{n-1}a_{n-2}\ldots a_1a_0\,,\; a_{-1}a_{-2}\ldots a_{-m})_b$$

може бути строго математично представлене у **розгорнутій формі (степеневому ряді)** [1]:

$$N = \sum_{k=-m}^{n-1} a_k \cdot b^k = a_{n-1}b^{n-1} + a_{n-2}b^{n-2} + \cdots + a_0 b^0 + a_{-1}b^{-1} + a_{-2}b^{-2} + \cdots + a_{-m}b^{-m}$$

де:

- b — основа системи числення;
- $a_k \in A_b$ — цифри числа в k-му розряді ($0 \le a_k \le b-1$);
- n — кількість розрядів у цілій частині числа;
- m — кількість розрядів у дробовій частині числа;
- k — індекс розряду ($k \ge 0$ для цілої частини, $k < 0$ для дробової частини).

Приклад розгорнутої форми для десяткового числа:

$$428{,}35_{10} = 4 \cdot 10^2 + 2 \cdot 10^1 + 8 \cdot 10^0 + 3 \cdot 10^{-1} + 5 \cdot 10^{-2}$$
$$= 400 + 20 + 8 + 0{,}3 + 0{,}05 = 428{,}35$$

Ця формула є універсальною: вона дозволяє не лише зрозуміти будову числа в будь-якій системі, але й служить прямим алгоритмічним методом переводу будь-якого числа з довільної системи в десяткову.

РОЗДІЛ 2. ОСНОВНІ КОМП'ЮТЕРНІ СИСТЕМИ ЧИСЛЕННЯ

Система числення	Основа (b)	Алфавіт цифр	Префікс у мовах програмування	Призначення в комп'ютерних системах
Двійкова (Binary)	2	$\{0, 1\}$	`0b` (наприклад, `0b1011`)	Фізичний рівень: сигнали на шинах, АЛП, регістри процесора
Вісімкова (Octal)	8	$\{0, 1, 2, 3, 4, 5, 6, 7\}$	`0o` або `0` (наприклад, `0o755`)	Права доступу в POSIX/Linux, старі 12/24/36-бітні архітектури (PDP)
Десяткова (Decimal)	10	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$	Відсутній (за замовчуванням)	Людино-машинний інтерфейс, введення та виведення результатів
Шістнадцяткова (Hex)	16	$\{0, \ldots, 9, \mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}, \mathrm{F}\}$	`0x` (наприклад, `0x7FFF`)	Адреси пам'яті, hex-дампи, кольори веб-палітри, коди інструкцій

2.1. Двійкова система числення (Binary) як основа цифрової електроніки

Двійкова система числення має основу $b = 2$ та використовує мінімально можливий позиційний алфавіт: $A_2 = \{0, 1\}$. Цифра двійкового числа називається **бітом** (від англ. *binary digit*).

Причини абсолютної домінації двійкової системи в ЕОМ:

1. **Фізична надійність і простота елементної бази.** Електронна схема повинна стійко функціонувати в умовах виробничих допусків, змін напруги живлення, коливань температури та паразитних наведень. Двійковий сигнал реалізується елементарним перемикачем (транзистором):
    - Стан закритого ключа (струм не тече, напруга близька до $0\,\mathrm{В}$) інтерпретується як логічний нуль (0).
    - Стан відкритого ключа (струм протікає, напруга відповідає шині живлення, наприклад $+1{,}2\,\mathrm{В}$, $+3{,}3\,\mathrm{В}$ або $+5\,\mathrm{В}$) інтерпретується як логічна одиниця (1).

Між цими рівнями існує широкий діапазон невизначеності (захисний зазор), що забезпечує колосальну завадостійкість схеми [4].

2. **Ізоморфізм двійкової арифметики та булевої алгебри.** У 1937 році американський інженер і математик Клод Шеннон у своїй фундаментальній праці довів, що операції двійкової математики повністю тотожні законам математичної логіки, сформульованим Джорджем Булем у 1854 році [9]. Будь-яка складна операція (додавання, віднімання, порівняння) зводиться до найпростіших логічних функцій: **І (AND)**, **АБО (OR)**, **НІ (NOT)** та **ВИКЛЮЧНЕ АБО (XOR)**. Це дозволило проєктувати процесори суто аналітично за допомогою таблиць істинності та карт Карно.
3. **Тривіальність правил арифметики.** Таблиці додавання та множення для двійкової системи складаються всього з чотирьох рядків:

$+$	0	1
0	0	1
1	1	10_2

$\times$	0	1
0	0	0
1	0	1

Таблиця множення зводиться до логічного кон'юнктивного елемента `AND`, а додавання одного розряду — до елемента `XOR` із формуванням переносу [4].

2.2. Вісімкова (Octal) та шістнадцяткова (Hexadecimal) системи: компактне подання машинних кодів та адрес

Хоча комп'ютер оперує виключно бітами, для людини-розробника робота з двійковими рядками становить значні труднощі. Розглянемо приклад: 32-розрядна адреса пам'яті виглядає як: `11011111000100001111110010100101_2`

Такий запис практично неможливо візуально сприйняти, запам'ятати чи перевірити на помилки. Саме для вирішення цієї проблеми застосовуються системи числення, основи яких є степенями двійки:

- **Вісімкова система:** $8 = 2^3$. Кожна вісімкова цифра замінює рівно **3 двійкові цифри (тріаду)**.
- **Шістнадцяткова система:** $16 = 2^4$. Кожна шістнадцяткова цифра замінює рівно **4 двійкові цифри (тетраду, або ніббл)**.

Шістнадцяткова система числення

В алфавіті шістнадцяткової системи перші десять цифр позначаються арабськими знаками $0..9$, а наступні шість значень — літерами латинського алфавіту:

$$\mathrm{A}_{16} = 10_{10}, \quad \mathrm{B}_{16} = 11_{10}, \quad \mathrm{C}_{16} = 12_{10}, \quad \mathrm{D}_{16} = 13_{10}, \quad \mathrm{E}_{16} = 14_{10}, \quad \mathrm{F}_{16} = 15_{10}$$

Перевага шістнадцяткової системи полягає в тому, що **один байт (8 біт) завжди записується рівно двома шістнадцятковими цифрами** в діапазоні від `0x00` (0) до `0xFF` (255).

Згадана вище 32-бітна адреса в шістнадцятковому поданні набуває лаконічного та легкого для читання вигляду: `0xDF10FCA5`

Основні сфери застосування шістнадцяткової системи:

1. **Адресація пам'яті:** Усі вказівники та адреси регістрів в операційних системах [2] та зневаджувачах (GDB, IDA Pro, x64dbg) виводяться у шістнадцятковій формі (наприклад, `0x7FFF5FBFF840`).
2. **Кодування кольорів (Web / 24-bit TrueColor):** Колір у форматі `#RRGGBB` розбивається на 3 канали по 8 біт: `#FF5733` означає червоний $\mathrm{FF}_{16} = 255_{10}$, зелений $57_{16} = 87_{10}$ та синій $33_{16} = 51_{10}$.
3. **Шістнадцяткові дампи пам'яті (Hex Dumps):** Зручний побайтовий перегляд бінарних файлів, прошивок мікроконтролерів та мережевих пакетів.
4. **Мережева адресація:** MAC-адреси мережевих адаптерів (`00:1A:2B:3C:4D:5E`) та адреси протоколу IPv6 (`2001:0db8:85a3:0000:0000:8a2e:0370:7334`).

РОЗДІЛ 3. АЛГОРИТМИ ПЕРЕВОДУ ЧИСЕЛ ТА ДВІЙКОВА АРИФМЕТИКА

3.1. Переведення цілих та дробових чисел між десятковою та іншими системами. Нескінченні двійкові дроби та стандарт IEEE 754

1. Переведення з довільної позиційної системи в десяткову

Здійснюється безпосереднім розгортанням числа в степеневий ряд або за оптимізованою **схемою Горнера**.

Приклад 1 (Степеневий ряд для цілого двійкового числа): Перевести 110101_2 у десяткову систему:

$$110101_2 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

$$= 32 + 16 + 0 + 4 + 0 + 1 = 53_{10}$$

Приклад 2 (Метод Горнера для цілих чисел): Схема Горнера мінімізує кількість операцій множення за рахунок дужкового групування [1]:

$$S_0 = a_{n-1}, \quad S_j = S_{j-1} \cdot b + a_{n-1-j}$$

Для числа 110101_2:

- Крок 1: 1
- Крок 2: $1 \cdot 2 + 1 = 3$
- Крок 3: $3 \cdot 2 + 0 = 6$
- Крок 4: $6 \cdot 2 + 1 = 13$
- Крок 5: $13 \cdot 2 + 0 = 26$

- Крок 6: $26 \cdot 2 + 1 = 53_{10}$

Приклад 3 (Степеневий ряд для дробового двійкового числа): Перевести $0{,}1011_2$ у десяткову систему:

$$0{,}1011_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 1 \cdot 2^{-4}$$

$$= 0{,}5 + 0 + 0{,}125 + 0{,}0625 = 0{,}6875_{10}$$

---

2. Переведення з десяткової системи в довільну позиційну

Оскільки ціла та дробова частини підпорядковані різній динаміці ваг розрядів, переведення виконується **окремо для цілої та окремо для дробової частини**.

А. Алгоритм послідовного ділення для цілої частини:

1. Початкове число ділиться націло на нову основу b.
2. Залишок від ділення ($0 \le r < b$) фіксується як поточний розряд.
3. Отримана частка знову ділиться на b.
4. Процес припиняється, коли частка стає рівною 0.
5. **Результат записується зі знайдених залишків у зворотному напрямку — від останнього залишку до першого.**

Приклад 4: Перевести число 157_{10} у двійкову систему:

- $157/2 = 78$ (залишок 1 — молодший біт LSB)
- $78/2 = 39$ (залишок 0)
- $39/2 = 19$ (залишок 1)
- $19/2 = 9$ (залишок 1)
- $9/2 = 4$ (залишок 1)
- $4/2 = 2$ (залишок 0)
- $2/2 = 1$ (залишок 0)
- $1/2 = 0$ (залишок 1 — старший біт MSB)

Виписуємо залишки від низу до верху: $157_{10} = 10011101_2$.

Б. Алгоритм послідовного множення для правильного дробу [7]:

1. Дробова частина множиться на нову основу b.
2. Ціла частина отриманого добутку фіксується як черговий розряд дробу після коми.
3. Дробова частина, що залишилася, знову множиться на b.
4. Алгоритм зупиняється, коли дробова частина стає рівною нулю (скінченний дріб) або коли досягнуто необхідної точності (ліміту розрядності).
5. **Цифри дробу виписуються в прямому порядку — згори донизу.**

Приклад 5 (Скінченний двійковий дріб): Перевести $0{,}625_{10}$ у двійкову систему:

- $0{,}625 \times 2 = \mathbf{1}{,}25 \implies$ фіксуємо цифру 1, залишок $0{,}25$

- $0{,}25 \times 2 = \mathbf{0}{,}50 \implies$ фіксуємо цифру 0, залишок $0{,}50$
- $0{,}50 \times 2 = \mathbf{1}{,}00 \implies$ фіксуємо цифру 1, залишок $0{,}00$ (зупинка)

Виписуємо цифри згори вниз: $0{,}625_{10} = 0{,}101_2$.

Об’єднуючи результати Прикладів 4 та 5, маємо:

$$157{,}625_{10} = 10011101{,}101_2$$

В. Нескінченні періодичні двійкові дроби та похибки обчислень (стандарт IEEE 754) [5]

У теорії чисел доведено фундаментальну теорему: нескоротний дріб p/q подається у позиційній системі числення з основою b скінченним дробом тоді й тільки тоді, коли всі прості дільники знаменника q входять до розкладу основи b на прості множники.

Для десяткової системи ($b = 10 = 2 \times 5$) знаменник може містити лише степені двійки та п’ятірки (наприклад, $1/2 = 0{,}5$, $1/4 = 0{,}25$, $1/5 = 0{,}2$, $1/10 = 0{,}1$). Проте для двійкової системи ($b = 2$) єдиним простим дільником є число 2. Отже, **будь-який десятковий дріб, знаменник якого ділиться на 5 (зокрема $0{,}1 = 1/10$ або $0{,}2 = 1/5$), перетворюється у двійковій системі на нескінченний періодичний дріб**.

Приклад 6 (Переведення десяткового числа $0{,}1_{10}$ у двійкову форму):

- $0{,}1 \times 2 = \mathbf{0}{,}2 \implies 0$
- $0{,}2 \times 2 = \mathbf{0}{,}4 \implies 0$
- $0{,}4 \times 2 = \mathbf{0}{,}8 \implies 0$
- $0{,}8 \times 2 = \mathbf{1}{,}6 \implies 1$
- $0{,}6 \times 2 = \mathbf{1}{,}2 \implies 1$
- $0{,}2 \times 2 = \mathbf{0}{,}4 \implies 0$ *(повторення значення $0{,}2$ — початок періоду)*

Отримуємо нескінченний періодичний двійковий дріб:

$$0{,}1_{10} = 0{,}0001100110011\ldots_2 = 0{,}0(0011)_2$$

Аналогічно:

$$0{,}2_{10} = 0{,}001100110011\ldots_2 = 0{,}(0011)_2$$

Вплив на архітектуру ЕОМ та стандарт IEEE 754:

У сучасних мікропроцесорах дійсні числа зберігаються відповідно до міжнародного стандарту **IEEE 754-2019** у форматі чисел з рухомою комою (Floating-Point):

$$N = (-1)^s \times (1 + M) \times 2^{E - \text{bias}}$$

де розрядна сітка має суворо фіксовану довжину:

- для формату одинарної точності (32 біти, тип `float`) під мантису відведено 23 біти;
- для формату подвійної точності (64 біти, тип `double`) під мантису відведено 52 біти.

Оскільки нескінченний період $0{,}0(0011)_2$ фізично неможливо розмістити у скінченній кількості бітів, значення на 53-му біті обрізається та заокруглюється до найближчого парного

значення. Внаслідок цього виникає неминуча апаратна похибка представлення:

$$0{,}1_{10} \approx 0{,}1000000000000000055511151231 26_{10}$$

$$0{,}2_{10} \approx 0{,}2000000000000000111022302462 52_{10}$$

При сумуванні цих величин у процесорі отримуємо хрестоматійний парадокс цифрових обчислень:

$$0{,}1 + 0{,}2 = 0{,}30000000000000004440892098500 6 \neq 0{,}3$$

Це демонструє, що похибка округлення є не програмним багом конкретної мови програмування (JavaScript, Python чи C++), а фундаментальною властивістю двійкового машинного подання чисел. У банківських системах та фінансовому інжинірингу для уникнення накопичення таких похибок категорично заборонено використовувати стандартні типи з рухомою комою; замість них застосовують десяткову арифметику фіксованої точки (наприклад, тип `Decimal` або двійково-десятковий код BCD).

3.2. Швидкий перехід між системами з основами $2, 8, 16$ (метод тріад і тетрад) [7, 8]

Оскільки основи систем 8 та 16 є цілими степенями двійки ($8 = 2^3, 16 = 2^4$), перехід між двійковою та цими системами здійснюється **без проміжного переведення в десяткову форму** шляхом прямого групування цифр.

Правило переведення з двійкової системи у шістнадцяткову:

1. Двійкове число розбивають на групи по **4 цифри (тетради)**:
    - для цілої частини — **від коми вліво**;
    - для дробової частини — **від коми вправо**.
2. Якщо крайня ліва або права група не містить 4 цифр, її доповнюють незначущими нулями (ліворуч для цілої частини, праворуч для дробової).
3. Кожна тетрада замінюється відповідною шістнадцятковою цифрою з таблиці.

Приклад 7: Перевести двійкове число $11010111100{,}101101_2$ у шістнадцяткову форму:

- Розбиваємо цілу частину вліво: `110` $\rightarrow$ доповнюємо до `0110`, далі `1011`, далі `1100`.
- Розбиваємо дробову частину вправо: `1011`, далі `01` $\rightarrow$ доповнюємо до `0100`.
- Маємо розбивку:

$$(\underbrace{0110}_{6}\ \underbrace{1011}_{B}\ \underbrace{1100}_{C}\,,\ \underbrace{1011}_{B}\ \underbrace{0100}_{4})_2 = \mathbf{6BC{,}B4_{16}}$$

3.3. Арифметичні операції у двійковій системі та машинне кодування чисел (прямий, обернений, додатковий коди)

1. Двійкове додавання та множення у стовпчик

А. Додавання у стовпчик

Додавання багаторозрядних двійкових чисел виконується порозрядно від молодших розрядів до старших. При додаванні $1_2 + 1_2$ сума в поточному розряді дорівнює 0, а в сусідній старший розряд генерується одиниця міжрозрядного переносу (**carry**) [4]:

$$\begin{array}{rl} & \overset{111000}{} \\ & 101101_2 \quad (45_{10}) \\ + & 011011_2 \quad (27_{10}) \\ \hline & 1001000_2 \quad (72_{10}) \end{array}$$

Перевірка в десятковій системі: $45 + 27 = 72$. Результат абсолютно точний.

Б. Множення у стовпчик

Множення у двійковій системі зводиться до повторюваних операцій формування часткових добутків за допомогою логічної операції кон'юнкції (`AND`), їх зсуву вліво відповідно до ваги розряду множника та наступного порозрядного сумування. Оскільки множення на двійкову одиницю дає саме множене, а на нуль — нульовий рядок, процес алгоритмічно набагато простіший за десятковий.

Приклад 8 (Двійкове множення у стовпчик): Перемножити числа 1101_2 (13_{10}) та 1011_2 (11_{10}):

$$\begin{array}{rl} & 1101_2 \quad (13) \\ \times & 1011_2 \quad (11) \\ \hline & 1101 \\ + & 1101 \\ + & 0000 \\ + & 1101 \\ \hline & 10001111_2 \quad (143_{10}) \end{array}$$

Перевірка в десятковій системі: $13 \times 11 = 143$.

В апаратній архітектурі процесора саме цей принцип зсуву та додавання реалізують спеціалізовані апаратні конвеєрні помножувачі та помножувачі за алгоритмом Бута (Booth's algorithm) [3].

---

2. Машинне представлення чисел зі знаком у розрядній сітці ЕОМ

У фізичній пам'яті комп'ютера відсутній окремий символ «мінус». Замість цього старший розряд (Most Significant Bit, MSB) фіксованої розрядної сітки процесора відводиться під **знак числа (sign bit)**:

- Біт знака $= 0 \implies$ число додатне;
- Біт знака $= 1 \implies$ число від'ємне.

В історії розвитку обчислювальної техніки застосовувалися три формати кодування цілих чисел зі знаком [2, 7].

А. Прямий код (Sign and Magnitude / Direct Code)

У прямому коді старший біт є знаковим, а в решті $N-1$ молодших розрядах розміщується двійковий модуль числа:

- для додатного числа: $[+X]_{\text{пр}} = 0\,,\ x_{n-2}\ldots x_0$;
- для від'ємного числа: $[-X]_{\text{пр}} = 1\,,\ x_{n-2}\ldots x_0$.

Наприклад, у 8-розрядній сітці:

$$[+5]_{\text{пр}} = \mathbf{0}0000101_2, \qquad [-5]_{\text{пр}} = \mathbf{1}0000101_2$$

Причини відмови від прямого коду:

1. **Проблема двох нулів:** виникає два різних машинні коди для нуля — $+0 = 00000000_2$ та $-0 = 10000000_2$. Це змушувало апаратуру виконувати зайві перевірки при кожному логічному порівнянні з нулем.
2. **Неможливість віднімання на суматорі:** пряме додавання додатного та від'ємного чисел у прямому коді дає хибний результат:

   $$[+5]_{\text{пр}} + [-5]_{\text{пр}} = 00000101_2 + 10000101_2 = 10001010_2 = -10_{10} \neq 0$$

   Для реалізації віднімання вимагалося конструювати окремий складний апаратний віднімач, що значно збільшувало кількість транзисторів і площу кристала.

Б. Обернений код (Ones' Complement / Inverse Code)

Для додатних чисел збігається з прямим кодом. Для від'ємних чисел формується шляхом **побітового інвертування (заперечення NOT)** усіх розрядів двійкового модуля числа (або всіх інформаційних бітів):

$$[-X]_{\text{об}} = [\bar{X}]$$

Наприклад, для 8-бітного числа $+5_{10} = 00000101_2$:

$$[-5]_{\text{об}} = \mathbf{1}1111010_2$$

Недоліки оберненого коду:

1. Проблема двох нулів залишилася нерозв'язаною: $+0 = 00000000_2$ та $-0 = 11111111_2$.
2. При виконанні арифметичних операцій виникає явище **кругового переносу (end-around carry)**: якщо перенос виникає у знаковому розряді, цю одиницю потрібно апаратно подавати зворотним зв'язком на вхід молодшого розряду суматора, що уповільнює тактову частоту процесора.

В. Додатковий код (Two's Complement) — сучасний інженерний стандарт

У додатковому коді додатні числа записуються у стандартному двійковому вигляді зі знаковим бітом 0, а для формування від'ємних чисел застосовується правило:

$$\boxed{[-X]_{\text{дод}} = [-X]_{\text{об}} + 1 = [\bar{X}] + 1}$$

тобто **інвертувати всі біти абсолютного значення числа та додати двійкову одиницю до молодшого розряду**.

Приклад 9: Записати число -5_{10} у 8-бітному додатковому коді:

1. Записуємо $+5_{10}$ у 8 бітах: `00000101`
2. Інвертуємо всі біти (обернений код): `11111010`
3. Додаємо одиницю: `11111010 + 1` = `11111011`

$$[-5]_{\text{дод}} = \mathbf{11111011_2} = \mathbf{0xFB}$$

Фундаментальні переваги додаткового коду:

1. **Єдине подання нуля:** код $+0$ є `00000000`. При спробі закодувати -0: інверсія дає `11111111`, додавання 1 породжує `1 00000000`. Одиниця переносу виходить за межі 8-бітної сітки й відкидається, залишаючи суворо `00000000`. «Від'ємний нуль» усунуто математично!
2. **Усунення операції віднімання з АЛП:** операція $A - B$ тотожна додаванню $A + [-B]_{\text{дод}}$. Увесь процесор будується навколо єдиного високошвидкісного суматора.

Приклад 10 (Віднімання в АЛП): Обчислити $12_{10} - 5_{10}$ у 8-розрядному АЛП:

- $+12_{10} = 00001100_2$
- $-5_{10} = 11111011_2$
- Виконуємо звичайне двійкове додавання:

$$\begin{array}{rl} & 00001100_2 \quad (12) \\ + & 11111011_2 \quad (-5) \\ \hline & \mathbf{1}\,00000111_2 \end{array}$$

Дев'ятий біт переносу відкидається апаратно.
Залишається результат: $00000111_2 = +7_{10}$. Обчислення виконано суматором без будь-яких додаткових блоків віднімання [3, 7]!

Г. Діапазон значень та переповнення розрядної сітки (Integer Overflow)

Для N-розрядної сітки чисел зі знаком у додатковому коді діапазон допустимих значень визначається нерівністю:

$$-2^{N-1} \le X \le 2^{N-1} - 1$$

Для 8-бітного числа: від -128 (10000000_2) до $+127$ (01111111_2). Асиметрія діапазону (від'ємних чисел рівно на одне більше) є прямою перевагою усунення коду «-0», завдяки чому комбінація `10000000` виділена під число -128.

Явище переповнення (Integer Overflow):

Якщо результат арифметичної операції перевищує максимальну величину, яку може вмістити розрядна сітка, біти суми проникають у знаковий розряд, змінюючи знак результату на протилежний. Наприклад, додавання двох додатних чисел $+120_{10}$ та $+15_{10}$ у 8 бітах:

$$01111000_2\ (+120) + 00001111_2\ (+15) = \mathbf{1}0000111_2 = -121_{10}$$

Для апаратної фіксації цього збою процесор виставляє спеціальний прапорець переповнення `OF` **(Overflow Flag)** у регістрі стану. Врахування переповнення цілих чисел є критичним правилом безпечного програмування системного ПЗ та смарт-контрактів.

ВИСНОВКИ

У ході теоретичного аналізу та практичних розрахунків у рефераті досягнуто наступних результатів:

1. **Еволюція систем числення:** Доведено, що непозиційні системи числення через відсутність нуля, громіздкість запису та відсутність єдиного алгоритму арифметичних операцій виявилися тупиковою гілкою для машинних обчислень. Позиційний принцип із поліноміальним розкладом за степенями основи b став базисом усіх сучасних цифрових архітектур.
2. **Двійкова система як фізичний імператив:** Обґрунтовано, що використання двійкової системи ($b = 2$) зумовлене фізичними властивостями напівпровідникових транзисторів, які мають два стабільні робочі стани з максимальною завадостійкістю та найменшим енергоспоживанням. Двійкова арифметика повністю еквівалентна булевій алгебрі, що дозволяє реалізовувати мікропроцесорні пристрої на базі простих логічних вентилів.
3. **Роль вісімкової та шістнадцяткової систем:** Встановлено, що вісімкова та шістнадцяткова системи не застосовуються всередині апаратури безпосередньо, а служать ущільненим людино-орієнтованим поданням двійкового коду. Завдяки кратності ступеням двійки (2^3 та 2^4) перехід між ними та двійковим кодом реалізується миттєво через заміну груп бітів (тріад і тетрад) без трудомістких ділень чи множень. Шістнадцяткова система є де-факто промисловим стандартом для адресації оперативної пам'яті, представлення кольорів та hex-дампів.
4. **Двійкова арифметика та додатковий код:** Продемонстровано, що застосування додаткового коду (Two's Complement) розв'язало дві фундаментальні проблеми обчислювальної техніки: усунуло дублювання «плюс/мінус нуля» (яке було властиве прямому й оберненому кодам) та дозволило повністю виключити з мікропроцесора фізичний блок апаратного віднімання, звівши його до звичайного додавання з інвертованим операндом.
5. **Практичне значення для інженерії ПЗ:** Розуміння скінченності розрядної сітки, природи двійкового переповнення (Integer Overflow) та виникнення похибок при переході дробових чисел (IEEE 754-2019) є критично необхідним знанням для розробки високонадійного програмного забезпечення, виключення критичних вразливостей пам'яті та створення ефективних алгоритмів низькорівневої оптимізації.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. **Knuth D. E.** The Art of Computer Programming. Volume 2: Seminumerical Algorithms. 3rd ed. Boston: Addison-Wesley, 1997. 784 p.
2. **Tanenbaum A. S., Austin T.** Structured Computer Organization. 6th ed. Boston: Pearson, 2012. 800 p.
3. **Patterson D. A., Hennessy J. L.** Computer Organization and Design: The Hardware/Software Interface. 6th ed. Cambridge: Morgan Kaufmann, 2020. 848 p.
4. **Harris D. M., Harris S. L.** Digital Design and Computer Architecture. 2nd ed. Waltham: Morgan Kaufmann, 2012. 712 p.
5. **IEEE Std 754-2019.** IEEE Standard for Floating-Point Arithmetic. New York: IEEE Computer Society, 2019. 84 p. DOI: 10.1109/IEEESTD.2019.8766229.
6. **ДСТУ ISO/IEC 2382:2015.** Інформаційні технології. Словник термінів (ISO/IEC 2382:2015, IDT). Київ: ДП «УкрНДНЦ», 2016. 344 с.
7. **Матвієнко М. П.** Архітектура комп’ютера: підручник. 2-ге вид., переробл. і доповн. Київ: Ліра-К, 2021. 544 с.
8. **Мельник А. О.** Архітектура комп’ютерів: підручник. Львів: Видавництво Національного університету «Львівська політехніка», 2008. 470 с.
9. **Shannon C. E.** A Symbolic Analysis of Relay and Switching Circuits. *Transactions of the American Institute of Electrical Engineers*. 1938. Vol. 57, no. 12. P. 713–723. DOI: 10.1109/T-AIEE.1938.5057767.